

```
subq $8,%rsp      Decrement stack pointer
movq %rbp, (%rsp)  Store %rbp on stack
```

它们之间的区别是在机器代码中 pushq 指令编码为 1 个字节，而上面那两条指令一共需要 8 个字节。图 3-9 中前两栏给出的是，当 %rsp 为 0x108, %rax 为 0x123 时，执行指令 pushq %rax 的效果。首先 %rsp 会减 8，得到 0x100，然后将 0x123 存放到内存地址 0x100 处。

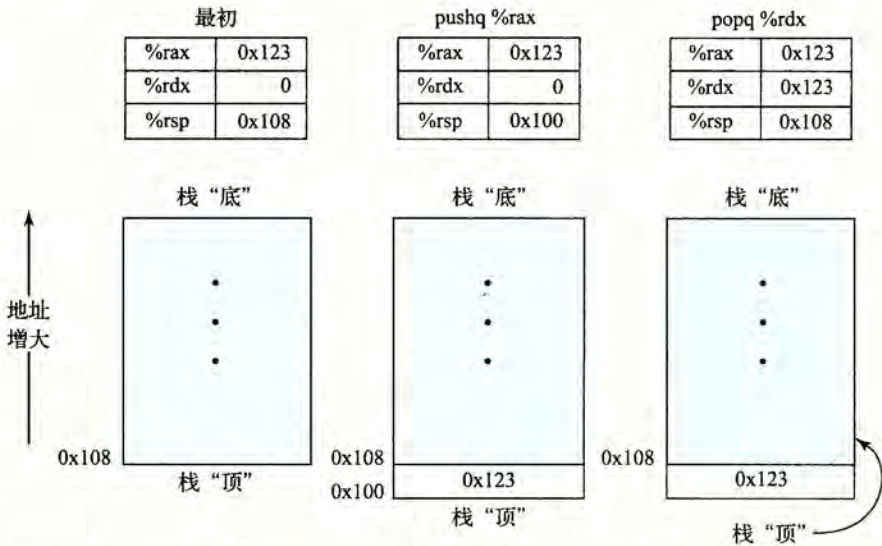


图 3-9 栈操作说明。根据惯例，我们的栈是倒过来画的，因而栈“顶”在底部。x86-64 中，栈向低地址方向增长，所以压栈是减小栈指针（寄存器 %rsp）的值，并将数据存放到内存中，而出栈是从内存中读数据，并增加栈指针的值

弹出一个四字的操作包括从栈顶位置读出数据，然后将栈指针加 8。因此，指令 popq %rax 等价于下面两条指令：

```
movq (%rsp), %rax    Read %rax from stack
addq $8, %rsp         Increment stack pointer
```

图 3-9 的第三栏说明的是在执行完 pushq 后立即执行指令 popq %rdx 的效果。先从内存中读出值 0x123，再写到寄存器 %rdx 中，然后，寄存器 %rsp 的值将增加回到 0x108。如图中所示，值 0x123 仍然会保持在内存位置 0x100 中，直到被覆盖（例如被另一条入栈操作覆盖）。无论如何，%rsp 指向的地址总是栈顶。

因为栈和程序代码以及其他形式的程序数据都是放在同一内存中，所以程序可以用标准的内存寻址方法访问栈内的任意位置。例如，假设栈顶元素是四字，指令 movq 8(%rsp), %rdx 会将第二个四字从栈中复制到寄存器 %rdx。

3.5 算术和逻辑操作

图 3-10 列出了 x86-64 的一些整数和逻辑操作。大多数操作都分成了指令类，这些指令类有各种带不同大小操作数的变种（只有 leaq 没有其他大小的变种）。例如，指令类 ADD 由四条加法指令组成：addb、addw、addl 和 addq，分别是字节加法、字加法、双字加法和四字加法。事实上，给出的每个指令类都有对这四种不同大小数据的指令。这些

操作被分为四组：加载有效地址、一元操作、二元操作和移位。二元操作有两个操作数，而一元操作有一个操作数。这些操作数的描述方法与 3.4 节中所讲的一样。

指令	效果	描述
<code>leaq S, D</code>	$D \leftarrow \&S$	加载有效地址
<code>INC D</code>	$D \leftarrow D + 1$	加1
<code>DEC D</code>	$D \leftarrow D - 1$	减1
<code>NEG D</code>	$D \leftarrow -D$	取负
<code>NOT D</code>	$D \leftarrow \sim D$	取补
<code>ADD S, D</code>	$D \leftarrow D + S$	加
<code>SUB S, D</code>	$D \leftarrow D - S$	减
<code>IMUL S, D</code>	$D \leftarrow D * S$	乘
<code>XOR S, D</code>	$D \leftarrow D \wedge S$	异或
<code>OR S, D</code>	$D \leftarrow D \vee S$	或
<code>AND S, D</code>	$D \leftarrow D \& S$	与
<code>SAL k, D</code>	$D \leftarrow D \ll k$	左移
<code>SHL k, D</code>	$D \leftarrow D \ll k$	左移（等同于SAL）
<code>SAR k, D</code>	$D \leftarrow D \gg_A k$	算术右移
<code>SHR k, D</code>	$D \leftarrow D \gg_L k$	逻辑右移

图 3-10 整数算术操作。加载有效地址(`leaq`)指令通常用来执行简单的算术操作。其余的指令是更加标准的一元或二元操作。我们用 $\gg_A$ 和 $\gg_L$ 来分别表示算术右移和逻辑右移。注意，这里的操作顺序与 ATT 格式的汇编代码中的相反

3.5.1 加载有效地址

加载有效地址(load effective address)指令 `leaq` 实际上是 `movq` 指令的变形。它的指令形式是从内存读数据到寄存器，但实际上它根本就没有引用内存。它的第一个操作数看上去是一个内存引用，但该指令并不是从指定的位置读入数据，而是将有效地址写入到目的操作数。在图 3-10 中我们用 C 语言的地址操作符 `&S` 说明这种计算。这条指令可以为后面的内存引用产生指针。另外，它还可以简洁地描述普通的算术操作。例如，如果寄存器 `%rdx` 的值为 x ，那么指令 `leaq 7(%rdx,%rdx,4),%rax` 将设置寄存器 `%rax` 的值为 $5x + 7$ 。编译器经常发现 `leaq` 的一些灵活用法，根本就与有效地址计算无关。目的操作数必须是一个寄存器。

为了说明 `leaq` 在编译出的代码中的使用，看看下面这个 C 程序：

```
long scale(long x, long y, long z) {
    long t = x + 4 * y + 12 * z;
    return t;
}
```

编译时，该函数的算术运算以三条 `leaq` 指令实现，就像右边注释说明的那样：

```
long scale(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
scale:
leaq    (%rdi,%rsi,4), %rax    x + 4*y
leaq    (%rdx,%rdx,2), %rdx    z + 2*z = 3*z
leaq    (%rax,%rdx,4), %rax    (x+4*y) + 4*(3*z) = x + 4*y + 12*z
ret
```


leaq 指令能执行加法和有限形式的乘法，在编译如上简单的算术表达式时，是很有用处的。

 **练习题 3.6** 假设寄存器 %rax 的值为 x, %rcx 的值为 y。填写下表，指明下面每条汇编代码指令存储在寄存器 %rdx 中的值：

表达式	结果
leaq 6(%ax), %rdx	
leaq (%rax, %rcx), %rdx	
leaq (%rax, %rcx, 4), %rdx	
leaq 7(%rax, %rax, 8), %rdx	
leaq 0xA(, %rcx, 4), %rdx	
leaq 9(%rax, %rcx, 2), %rdx	

 **练习题 3.7** 考虑下面的代码，我们省略了被计算的表达式：

```
long scale2(long x, long y, long z) {
    long t = _____;
    return t;
}
```

用 GCC 编译实际的函数得到如下的汇编代码：

```
long scale2(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
scale2:
    leaq    (%rdi,%rdi,4), %rax
    leaq    (%rax,%rsi,2), %rax
    leaq    (%rax,%rdx,8), %rax
    ret
```

填写出 C 代码中缺失的表达式。

3.5.2 一元和二元操作

第二组中的操作是一元操作，只有一个操作数，既是源又是目的。这个操作数可以是一个寄存器，也可以是一个内存位置。比如说，指令 `incq(%rsp)` 会使栈顶的 8 字节元素加 1。这种语法让人想起 C 语言中的加 1 运算符(++)和减 1 运算符(--)。

第三组是二元操作，其中，第二个操作数既是源又是目的。这种语法让人想起 C 语言中的赋值运算符，例如 `x-=y`。不过，要注意，源操作数是第一个，目的操作数是第二个，对于不可交换操作来说，这看上去很奇特。例如，指令 `subq %rax, %rdx` 使寄存器 %rdx 的值减去 %rax 中的值。(将指令解读成“从 %rdx 中减去 %rax”会有所帮助。)第一个操作数可以是立即数、寄存器或是内存位置。第二个操作数可以是寄存器或是内存位置。注意，当第二个操作数为内存地址时，处理器必须从内存读出值，执行操作，再把结果写回内存。

 **练习题 3.8** 假设下面的值存放在指定的内存地址和寄存器中：

地址	值
0x100	0xFF
0x108	0xAB
0x110	0x13
0x118	0x11

寄存器	值
%rax	0x100
%rcx	0x1
%rdx	0x3

填写下表，给出下面指令的效果，说明将被更新的寄存器或内存位置，以及得到的值：

指令	目的	值
<code>addq %rcx, (%rax)</code>		
<code>subq %rdx, 8(%rax)</code>		
<code>imulq \$16, (%rax, %rdx, 8)</code>		
<code>incq 16(%rax)</code>		
<code>decq %rcx</code>		
<code>subq %rdx, %rax</code>		

3.5.3 移位操作

最后一组是移位操作，先给出移位量，然后第二项给出的是要移位的数。可以进行算术和逻辑右移。移位量可以是一个立即数，或者放在单字节寄存器%cl中。（这些指令很特别，因为只允许以这个特定的寄存器作为操作数。）原则上来说，1个字节的移位量使得移位量的编码范围可以达到 $2^8 - 1 = 255$ 。x86-64中，移位操作对 w 位长的数据值进行操作，移位量是由%cl寄存器的低 m 位决定的，这里 $2^m = w$ 。高位会被忽略。所以，例如当寄存器%cl的十六进制值为0xFF时，指令salb会移7位，salw会移15位，sall会移31位，而salq会移63位。

如图3-10所示，左移指令有两个名字：SAL和SHL。两者的效果是一样的，都是将右边填上0。右移指令不同，SAR执行算术移位（填上符号位），而SHR执行逻辑移位（填上0）。移位操作的目的操作数可以是一个寄存器或是一个内存位置。图3-10中用 $\gg_A$ （算术）和 $\gg_L$ （逻辑）来表示这两种不同的右移运算。

 **练习题 3.9** 假设我们想生成以下C函数的汇编代码：

```
long shift_left4_rightn(long x, long n)
{
    x <<= 4;
    x >>= n;
    return x;
}
```

下面这段汇编代码执行实际的移位，并将最后的结果放在寄存器%rax中。此处省略了两条关键的指令。参数 x 和 n 分别存放在寄存器%rdi和%rsi中。

```
long shift_left4_rightn(long x, long n)
x in %rdi, n in %rsi
shift_left4_rightn:
    movq    %rdi, %rax    Get x
                        x <<= 4
    movl    %esi, %ecx    Get n (4 bytes)
                        x >>= n
```

根据右边的注释，填出缺失的指令。请使用算术右移操作。

3.5.4 讨论

我们看到图3-10所示的大多数指令，既可以用于无符号运算，也可以用于补码运算。

只有右移操作要求区分有符号和无符号数。这个特性使得补码运算成为实现有符号整数运算的一种比较好的方法的原因之一。

图 3-11 给出了一个执行算术操作的函数示例，以及它的汇编代码。参数 x 、 y 和 z 初始时分别存放在内存 `%rdi`、`%rsi` 和 `%rdx` 中。汇编代码指令和 C 源代码行对应很紧密。第 2 行计算 $x \wedge y$ 的值。指令 3 和 4 用 `leaq` 和移位指令的组合来实现表达式 $z * 48$ 。第 5 行计算 $t1$ 和 `0x0F0F0F0F` 的 AND 值。第 6 行计算最后的减法。由于减法的目的寄存器是 `%rax`，函数会返回这个值。

```
long arith(long x, long y, long z)
{
    long t1 = x ^ y;
    long t2 = z * 48;
    long t3 = t1 & 0x0F0F0F0F;
    long t4 = t2 - t3;
    return t4;
}
```

a) C语言代码

```
long arith(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
1  arith:
2      xorq    %rsi, %rdi          t1 = x ^ y
3      leaq    (%rdx,%rdx,2), %rax  3*z
4      salq    $4, %rax            t2 = 16 * (3*z) = 48*z
5      andl    $252645135, %edi    t3 = t1 & 0x0F0F0F0F
6      subq    %rdi, %rax          Return t2 - t3
7      ret
```

b) 汇编代码

图 3-11 算术运算函数的 C 语言和汇编代码

在图 3-11 的汇编代码中，寄存器 `%rax` 中的值先后对应于程序值 $3*z$ 、 $z * 48$ 和 $t4$ (作为返回值)。通常，编译器产生的代码中，会用一个寄存器存放多个程序值，还会在寄存器之间传送程序值。

 **练习题 3.10** 下面的函数是图 3-11a 中函数一个变种，其中有些表达式用空格替代：

```
long arith2(long x, long y, long z)
{
    long t1 = _____;
    long t2 = _____;
    long t3 = _____;
    long t4 = _____;
    return t4;
}
```

实现这些表达式的汇编代码如下：

```
long arith2(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
```

```
arith2:
    orq    %rsi, %rdi
    sarq   $3, %rdi
    notq   %rdi
    movq   %rdx, %rax
    subq   %rdi, %rax
    ret
```

基于这些汇编代码，填写 C 语言代码中缺失的部分。

 **练习题 3.11** 常常可以看见以下形式的汇编代码行：

```
xorq %rdx,%rdx
```

但是在产生这段汇编代码的 C 代码中，并没有出现 EXCLUSIVE-OR 操作。

- 解释这条特殊的 EXCLUSIVE-OR 指令的效果，它实现了什么有用的操作。
- 更直接地表达这个操作的汇编代码是什么？
- 比较同样一个操作的两种不同实现的编码字节长度。

3.5.5 特殊的算术操作

正如我们在 2.3 节中看到的，两个 64 位有符号或无符号整数相乘得到的乘积需要 128 位来表示。x86-64 指令集对 128 位(16 字节)数的操作提供有限的支持。延续字(2 字节)、双字(4 字节)和四字(8 字节)的命名惯例，Intel 把 16 字节的数称为八字(oct word)。图 3-12 描述的是支持产生两个 64 位数字的全 128 位乘积以及整数除法的指令。

指令	效果	描述
imulq S	$R[\%rdx] \leftarrow R[\%rax] \leftarrow S \times R[\%rax]$	有符号全乘法
mulq S	$R[\%rdx] \leftarrow R[\%rax] \leftarrow S \times R[\%rax]$	无符号全乘法
cltq	$R[\%rdx] \leftarrow R[\%rax] \leftarrow \text{符号扩展}(R[\%rax])$	转换为八字
idivq S	$R[\%rdx] \leftarrow R[\%rdx]; R[\%rax] \bmod S$ $R[\%rdx] \leftarrow R[\%rdx]; R[\%rax] \div S$	有符号除法
divq S	$R[\%rdx] \leftarrow R[\%rdx]; R[\%rax] \bmod S$ $R[\%rdx] \leftarrow R[\%rdx]; R[\%rax] \div S$	无符号除法

图 3-12 特殊的算术操作。这些操作提供了有符号和无符号数的全 128 位乘法和除法。
一对寄存器 %rdx 和 %rax 组成一个 128 位的八字

imulq 指令有两种不同的形式。其中一种，如图 3-10 所示，是 IMUL 指令类中的一种。这种形式的 imulq 指令是一个“双操作数”乘法指令。它从两个 64 位操作数产生一个 64 位乘积，实现了 2.3.4 和 2.3.5 节中描述的操作 $\times_{64}^u$ 和 $\times_{64}^s$ 。(回想一下，当将乘积截取到 64 位时，无符号乘和补码乘的位级行为是一样的。)

此外，x86-64 指令集还提供了两条不同的“单操作数”乘法指令，以计算两个 64 位值的全 128 位乘积——一个是无符号数乘法(mulq)，而另一个是补码乘法(imulq)。这两条指令都要求一个参数必须在寄存器 %rax 中，而另一个作为指令的源操作数给出。然后乘积存放在寄存器 %rdx(高 64 位)和 %rax(低 64 位)中。虽然 imulq 这个名字可以用于两个不同的乘法操作，但是汇编器能够通过计算操作数的数目，分辨出想用哪条指令。

下面这段 C 代码是一个示例，说明了如何从两个无符号 64 位数字 x 和 y 生成 128 位的乘积：


```
#include <inttypes.h>

typedef unsigned __int128 uint128_t;

void store_uprod(uint128_t *dest, uint64_t x, uint64_t y) {
    *dest = x * (uint128_t) y;
}
```

在这个程序中，我们显式地把 x 和 y 声明为 64 位的数字，使用文件 `inttypes.h` 中声明的定义，这是对标准 C 扩展的一部分。不幸的是，这个标准没有提供 128 位的值。所以我们只好依赖 GCC 提供的 128 位整数支持，用名字 `__int128` 来声明。代码用 `typedef` 声明定义了一个数据类型 `uint128_t`，沿用的 `inttypes.h` 中其他数据类型的命名规律。这段代码指明得到的乘积应该存放在指针 `dest` 指向的 16 字节处。

GCC 生成的汇编代码如下：

```
void store_uprod(uint128_t *dest, uint64_t x, uint64_t y)
dest in %rdi, x in %rsi, y in %rdx
1  store_uprod:
2      movq    %rsi, %rax          Copy x to multiplicand
3      mulq    %rdx                Multiply by y
4      movq    %rax, (%rdi)        Store lower 8 bytes at dest
5      movq    %rdx, 8(%rdi)       Store upper 8 bytes at dest+8
6      ret
```

可以观察到，存储乘积需要两个 `movq` 指令：一个存储低 8 个字节（第 4 行），一个存储高 8 个字节（第 5 行）。由于生成这段代码针对的是小端法机器，所以高位字节存储在大地址，正如地址 `8(%rdi)` 表明的那样。

前面的算术运算表（图 3-10）没有列出除法或取模操作。这些操作是由单操作数除法指令来提供的，类似于单操作数乘法指令。有符号除法指令 `idivl` 将寄存器 `%rdx`（高 64 位）和 `%rax`（低 64 位）中的 128 位数作为被除数，而除数作为指令的操作数给出。指令将商存储在寄存器 `%rax` 中，将余数存储在寄存器 `%rdx` 中。

对于大多数 64 位除法应用来说，除数也常常是一个 64 位的值。这个值应该存放在 `%rax` 中，`%rdx` 的位应该设置为全 0（无符号运算）或者 `%rax` 的符号位（有符号运算）。后面这个操作可以用指令 `cqto`[⊖] 来完成。这条指令不需要操作数——它隐含读出 `%rax` 的符号位，并将它复制到 `%rdx` 的所有位。

我们用下面这个 C 函数来说明 x86-64 如何实现除法，它计算了两个 64 位有符号数的商和余数：

```
void remdiv(long x, long y,
            long *qp, long *rp) {
    long q = x/y;
    long r = x%y;
    *qp = q;
    *rp = r;
}
```

该函数编译得到如下汇编代码：

⊖ 在 Intel 的文档中，这条指令叫做 `cqo`，这是指令的 ATT 格式名字和 Intel 名字无关的少数情况之一。


```

void remdiv(long x, long y, long *qp, long *rp)
x in %rdi, y in %rsi, qp in %rdx, rp in %rcx
1  remdiv:
2      movq    %rdx, %r8          Copy qp
3      movq    %rdi, %rax         Move x to lower 8 bytes of dividend
4      cqto    %rax               Sign-extend to upper 8 bytes of dividend
5      idivq   %rsi               Divide by y
6      movq    %rax, (%r8)        Store quotient at qp
7      movq    %rdx, (%rcx)       Store remainder at rp
8      ret

```

在上述代码中，必须首先把参数 `qp` 保存到另一个寄存器中(第2行)，因为除法操作要使用参数寄存器 `%rdx`。接下来，第3~4行准备被除数，复制并符号扩展 `x`。除法之后，寄存器 `%rax` 中的商被保存在 `qp`(第6行)，而寄存器 `%rdx` 中的余数被保存在 `rp`(第7行)。

无符号除法使用 `divq` 指令。通常，寄存器 `%rdx` 会事先设置为0。

 **练习题 3.12** 考虑如下函数，它计算两个无符号 64 位数的商和余数：

```

void uremdiv(unsigned long x, unsigned long y,
             unsigned long *qp, unsigned long *rp) {
    unsigned long q = x/y;
    unsigned long r = x%y;
    *qp = q;
    *rp = r;
}

```

修改有符号除法的汇编代码来实现这个函数。

3.6 控制

到目前为止，我们只考虑了直线代码的行为，也就是指令一条接一条顺序地执行。C语言中的某些结构，比如条件语句、循环语句和分支语句，要求有条件的执行，根据数据测试的结果来决定操作执行的顺序。机器代码提供两种基本的低级机制来实现有条件的行为：测试数据值，然后根据测试的结果来改变控制流或者数据流。

与数据相关的控制流是实现有条件行为的更一般和更常见的方法，所以我们先来介绍它。通常，C语言中的语句和机器代码中的指令都是按照它们在程序中出现的次序，顺序执行的。用 `jump` 指令可以改变一组机器代码指令的执行顺序，`jump` 指令指定控制应该被传递到程序的某个其他部分，可能是依赖于某个测试的结果。编译器必须产生构建在这种低级机制基础之上的指令序列，来实现C语言的控制结构。

本文会先涉及实现条件操作的两种方式，然后描述表达循环和 `switch` 语句的方法。

3.6.1 条件码

除了整数寄存器，CPU还维护着一组单个位的条件码(condition code)寄存器，它们描述了最近的算术或逻辑操作的属性。可以检测这些寄存器来执行条件分支指令。最常用的条件码有：

CF：进位标志。最近的操作使最高位产生了进位。可用来检查无符号操作的溢出。

ZF：零标志。最近的操作得出的结果为0。

SF：符号标志。最近的操作得到的结果为负数。

OF：溢出标志。最近的操作导致一个补码溢出——正溢出或负溢出。